

ITBA · SISTEMAS DE INTELIGENCIA ARTIFICIAL

Trabajo Práctico 1 Métodos de Búsqueda

Ejercicio 1 — 8-puzzle

Ejercicio 2 — Sokoban

EJERCICIO 1

8-puzzle

Estado · heurísticas admisibles · método de búsqueda

5	7	3
8	2	
1	6	4

¿Qué estructura de estado usaríamos?

Una tupla de nueve enteros, leyendo el tablero por filas. El 0 es el hueco

5	7	3
8	2	
1	6	4

estado = (5, 7, 3, 8, 2, 0, 1, 6, 4)

Inmutable

No se modifica después de ser creada

Hasheable

Entra en un set: chequear repetidos
cuesta $O(1)$ en vez de $O(n)$.

Comparación barata

Devuelve un bool y corta en la primera
diferencia.

Qué descartamos: la *matriz* 3×3 , más legible al imprimir pero hay que aplanarla para hashearla; y el *arreglo* de NumPy, no hasheable y cuya igualdad devuelve un array de bools.

Los tres tableros solución

Nunca son alcanzables los tres

	Tablero	Inversiones	Paridad	Clase
G1	1 2 3 / 4 5 6 / 7 8 _	0	par	A
G2	3 2 1 / 6 5 4 / _ 8 7	7	impar	B
G3	3 6 _ / 2 5 8 / 1 4 7	12	par	A

Ningún movimiento legal cambia la paridad de las inversiones. Mover el hueco en horizontal no altera el orden; en vertical una ficha salta dos posiciones. Las $9! = 362.880$ permutaciones quedan partidas en **dos clases incomunicadas de 181.440 estados**.

G1 y G3 comparten clase; G2 es la otra. El tablero del enunciado tiene 17 inversiones: sólo llega a G2, en 23 movimientos.

Dos heurísticas admisibles no triviales

h_1 · Manhattan

Suma de distancias de cada ficha a su lugar

Sin contar el blanco.

Por qué es admisible: cada movimiento desplaza exactamente una ficha una casilla, así que baja la suma en 1 como máximo.

h_2 · Manhattan + conflicto lineal

+2 por cada par invertido en su fila o columna destino

Si dos fichas ya están en su línea final pero cruzadas, una tiene que salir y volver: dos movimientos que Manhattan no cuenta.

Sigue siendo admisible y domina a h_1 : $h_2 \geq h_1$ para todo estado.

¿Qué método usaríamos, y por qué?

A* con h_2 $f(n) = g(n) + h(n)$ Óptimo con heurística admisible, y la poda evita re-explorar estados.

BFS

Óptimo pero ciego: expande por capas y explota en memoria.

Greedy

Muy rápido, no óptimo: sólo mira $h(n)$ e ignora lo recorrido.

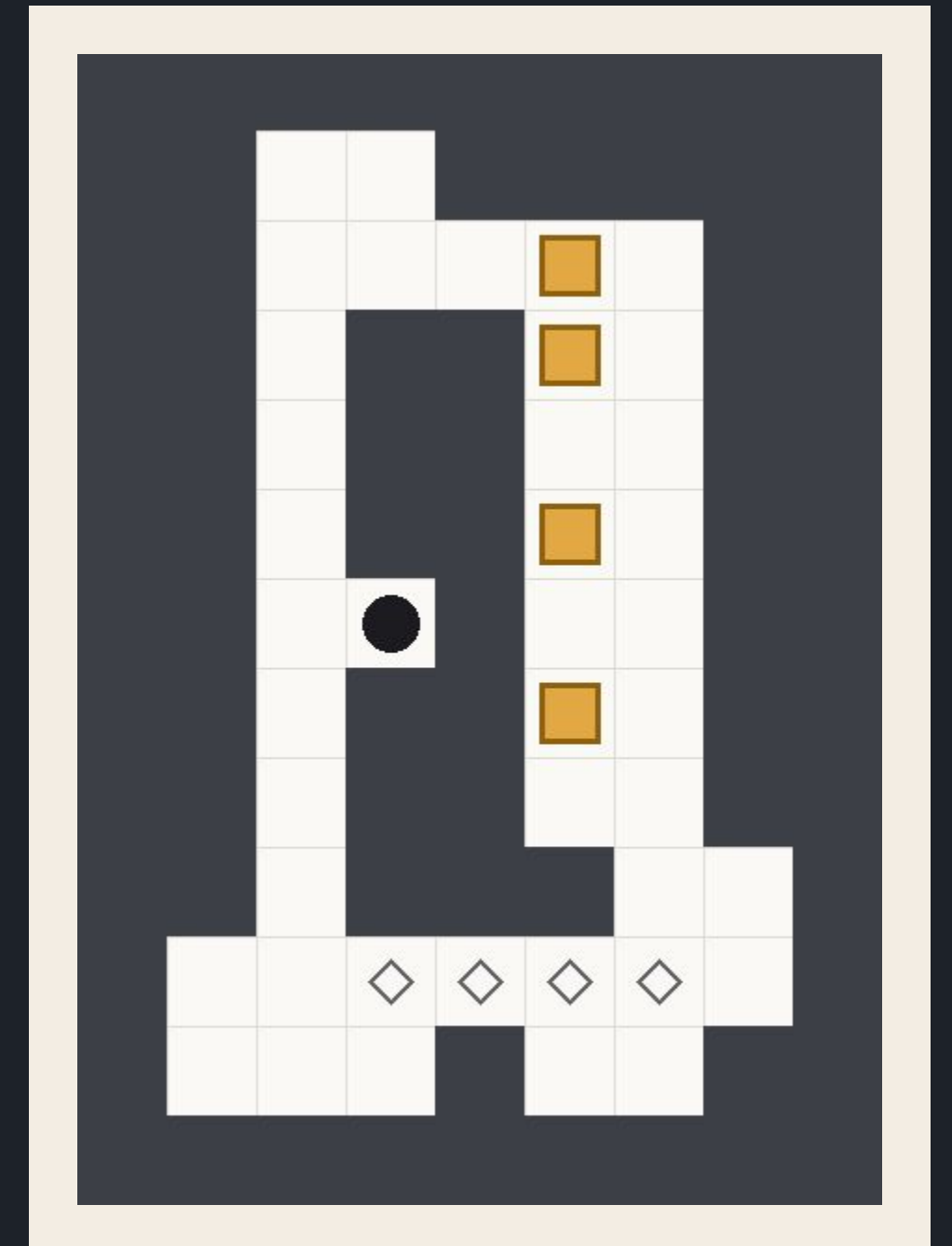
IDA*

Óptimo y con memoria mínima, pero re-expande en cada iteración.

EJERCICIO 2

Sokoban

Cinco niveles · cinco métodos · siete heurísticas



El problema bien definido

Los cinco componentes y nuestra decisión en cada uno

Componente	Nuestra decisión
Estado	(posición del jugador, frozenset de posiciones de las cajas)
Acciones	Cuatro movimientos del jugador (arriba, abajo, izquierda, derecha)
Transición	Mueve al jugador; si hay una caja en el destino, la empuja
Costo	1 por movimiento, empuje o no $\rightarrow g(n)$ = profundidad
Meta	Todas las cajas sobre metas

Estructura de estado

El estado - solo lo dinámico

El estado está compuesto por la posición del jugador y las posiciones de las cajas

jugador

frozenset(cajas)

`estado = ( jugador , frozenset(cajas) )`

El resto del tablero → estático → paredes, objetivos y tamaño del mapa se guardan en un objeto compartido: **no son parte del estado**

El grafo de búsqueda se define sobre esa tupla: dos configuraciones con el mismo jugador y las mismas cajas son el mismo nodo.

Los cinco niveles

Sacados de game-sokoban.com. Ninguno elegido al azar

N1 ABHT 02 · 01	N2 A.K.K. · 04	N3 Microban · 29	N4 A.K.K. · 02	N5 Sasquatch XII · 06
1 cajas	4 cajas	2 cajas	4 cajas	4 cajas
12 celdas	32 celdas	35 celdas	31 celdas	41 celdas
8 óptimo	45 óptimo	104 óptimo	70 óptimo	306 óptimo
Control de sanidad	El primero “real”, barato de iterar	El punto ciego de las heurísticas	Metas agrupadas → justifica el matching	El límite del método

PARTE B · SOKOBAN

Heurísticas

Cinco admisibles en escalera · dos no admisibles a propósito

Las siete heurísticas

Cada una arregla un defecto de la anterior

Heurística		Qué defecto arregla	Dónde se ve
h_1	cajas fuera de meta	Es el punto de partida, le da al resto de heurísticas un valor contra el que comparar	-
h_2	Σ Manhattan a la meta más cercana	h_1 toma muy pocos valores distintos	los cinco
h_3	matching óptimo con Manhattan	h_2 manda varias cajas a la misma meta	N4
h_4	matching con distancias de empuje	h_3 atraviesa paredes	N3, N5
h_5	h_4 + término del jugador	las anteriores ignoran al jugador	N3
$h_{\square_a}$	$2 \cdot h_4$	-	qué se rompe
$h_{\square_{a4}}$	$4 \cdot h_4$	-	qué se rompe

De contar cajas a medir distancias

h_1

Cajas fuera de meta

Admisible: cada caja mal ubicada necesita al menos un empuje.

Con cuatro cajas sólo toma cinco valores: 0, 1, 2, 3, 4. Casi no distingue nodos.

h_2

Σ Manhattan a la meta más cercana

Cada caja aporta su distancia a la meta libre más próxima: la función pasa a tener pendiente y muchos mas valores posibles.

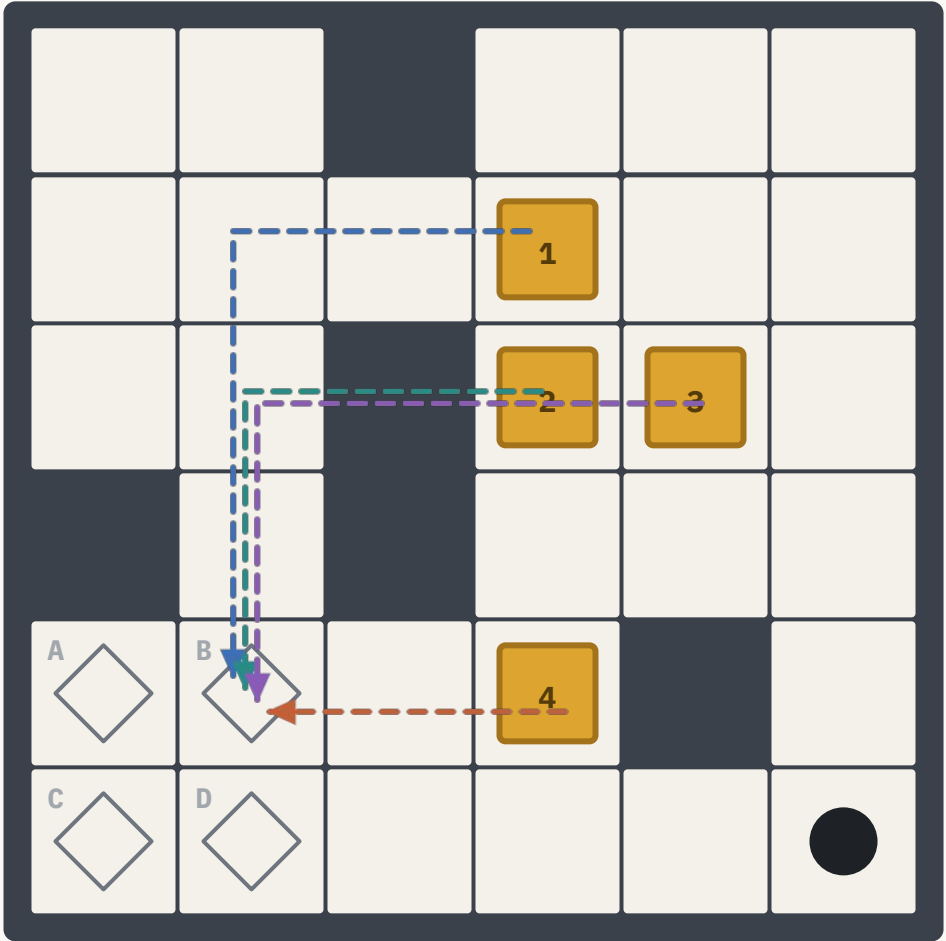
El problema de h_2 es que nada le puede mandar dos cajas a la misma meta.

h_3

Matching óptimo caja-meta

Una meta diferente asignada para cada caja

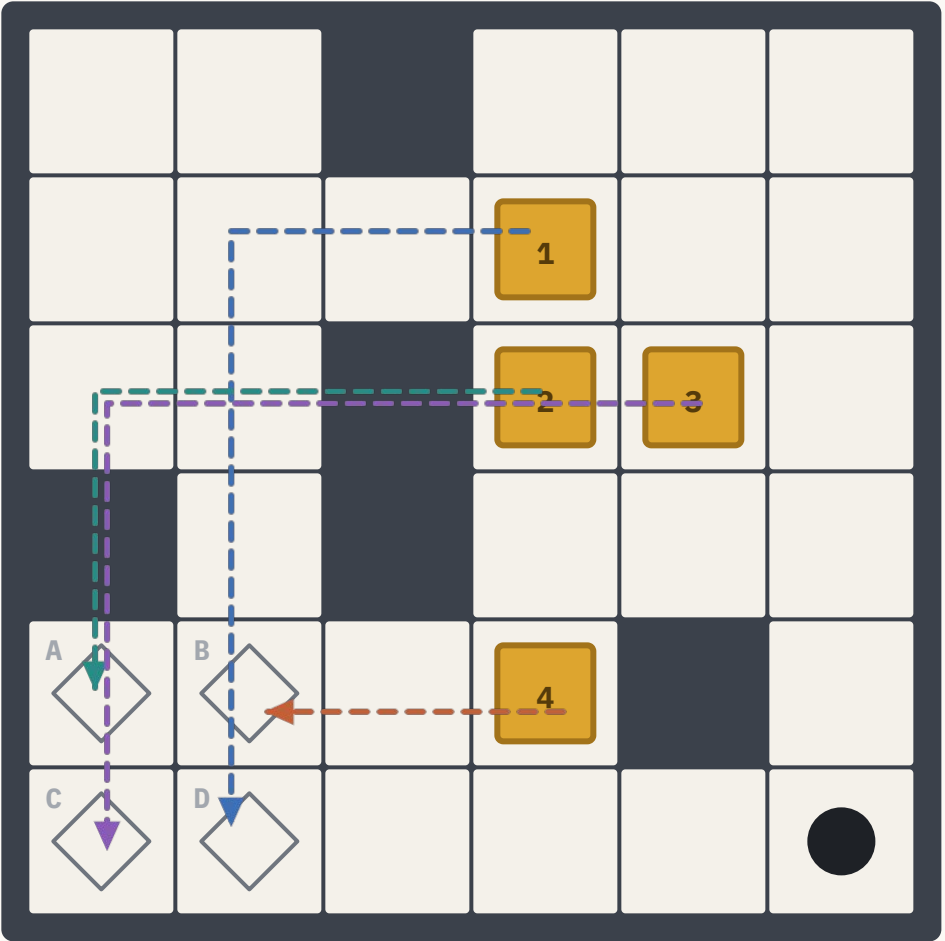
Con h_2 :



● Caja 1 → B	5	● Caja 3 → B	5
● Caja 2 → B	4	● Caja 4 → B	2

Las 4 cajas reclaman la meta B **16**

Con h_3 :



● Caja 1 → D	6	● Caja 3 → C	7
● Caja 2 → A	5	● Caja 4 → B	2

Emparejamiento 1 a 1 **20**

Lo que h_3 todavía no ve: las distancias son de línea recta, atraviesan paredes.

h_4

Matching con distancias de empuje

La misma asignación, pero con distancias que respetan las paredes

Reemplazamos Manhattan por la cantidad real de empujes necesarios para llevar una caja de una celda a una meta, precomputada sobre el tablero fijo. Se calcula una vez por nivel y después la heurística sólo hace consultas.

Dónde se ve — N3 y N5. Pasillos y recodos: la línea recta cruza paredes y miente por abajo; la distancia de empuje no.

Lo que h_4 todavía no ve: al jugador. Las cajas no se mueven solas.

h_5

h_4 más el término del jugador

El costo cuenta cada movimiento, no sólo los empujes. Si todavía queda una caja fuera de meta, el jugador tiene que caminar hasta ella: sumamos ese tramo, sin contarlo dos veces.

Dónde se ve — N3. Dos cajas lejos una de otra: la caminata entre empujes es la mitad del costo real.

h_5 es la heurística que usamos en las corridas comparativas de A^* y Greedy.

Dos heurísticas no admisibles, a propósito

Escalar h_4 para ver qué se rompe y qué se gana

$$h_{a4} = 2 \cdot h_4$$

Sobreestima moderada

A* deja de garantizar el óptimo, pero la frontera se ordena mucho más agresivamente.

$$h_{a4} = 4 \cdot h_4$$

Sobreestima fuerte

$g(n)$ queda prácticamente ignorado: A* se comporta como Greedy.

Sirven para medir el precio de la optimalidad: cuántos nodos se ahorran y cuánto se alarga la solución al aceptar un camino que no es el mejor.

PARTE B • SOKOBAN

Resultados

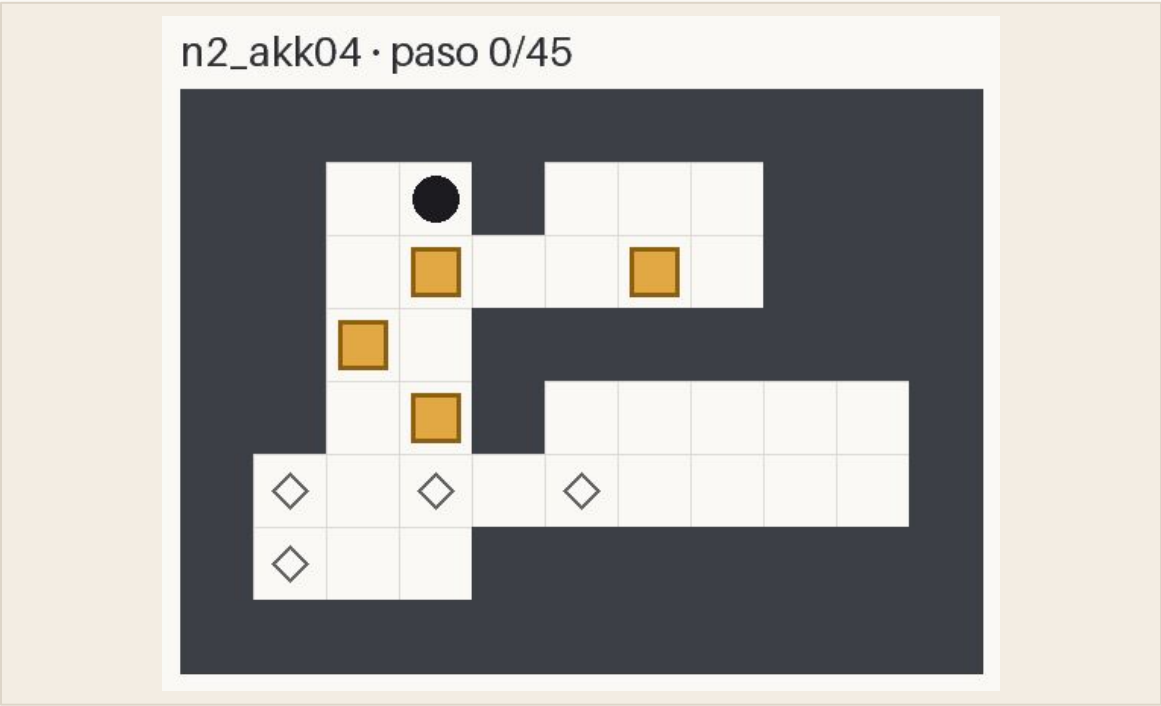
Cinco métodos sobre los cinco niveles

Soluciones

Los caminos óptimos ejecutados, uno por nivel



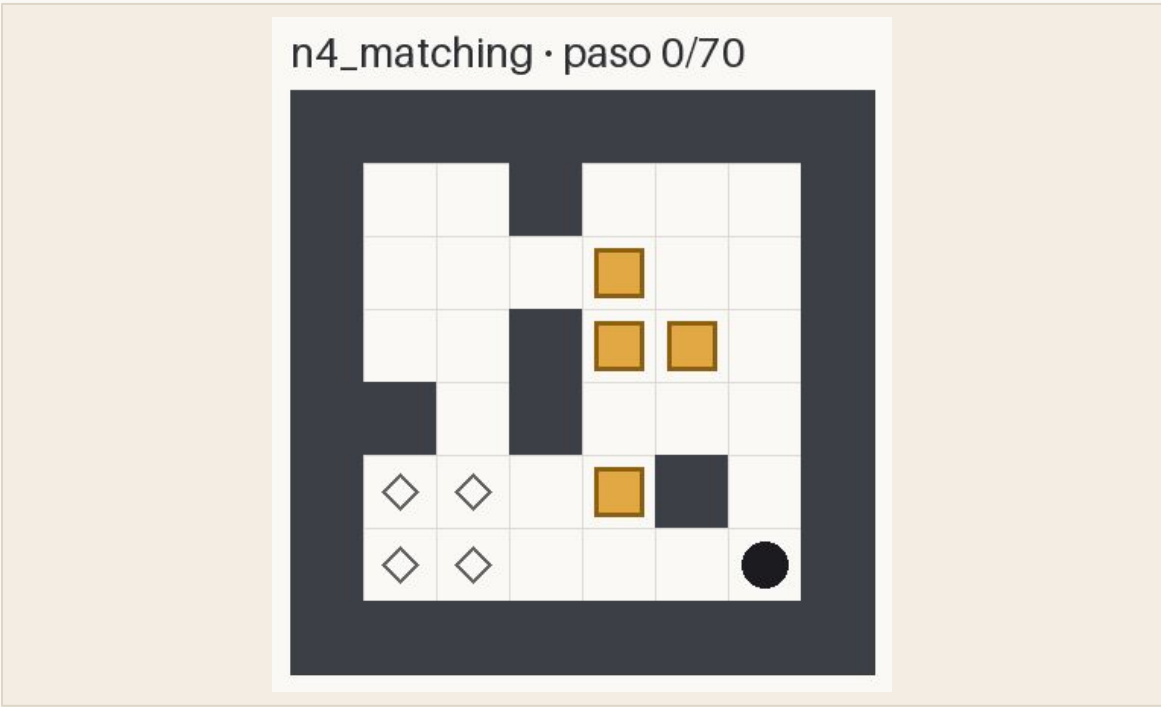
N1 · 8 mov



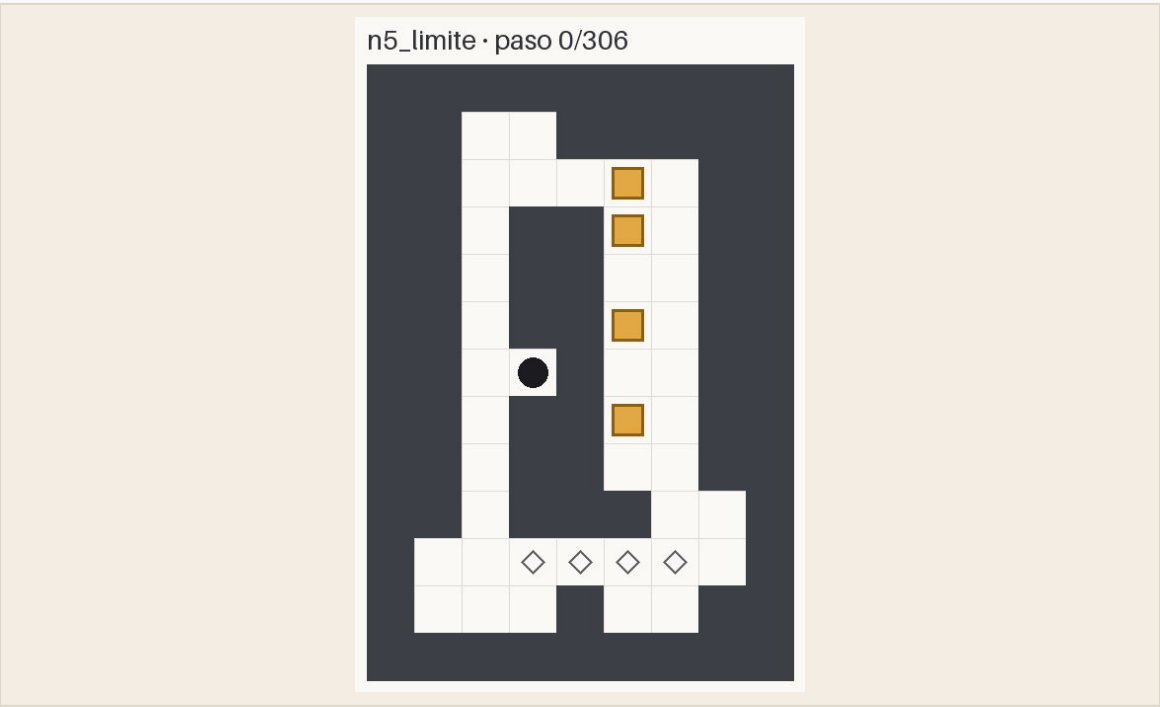
N2 · 45 mov



N3 · 104 mov

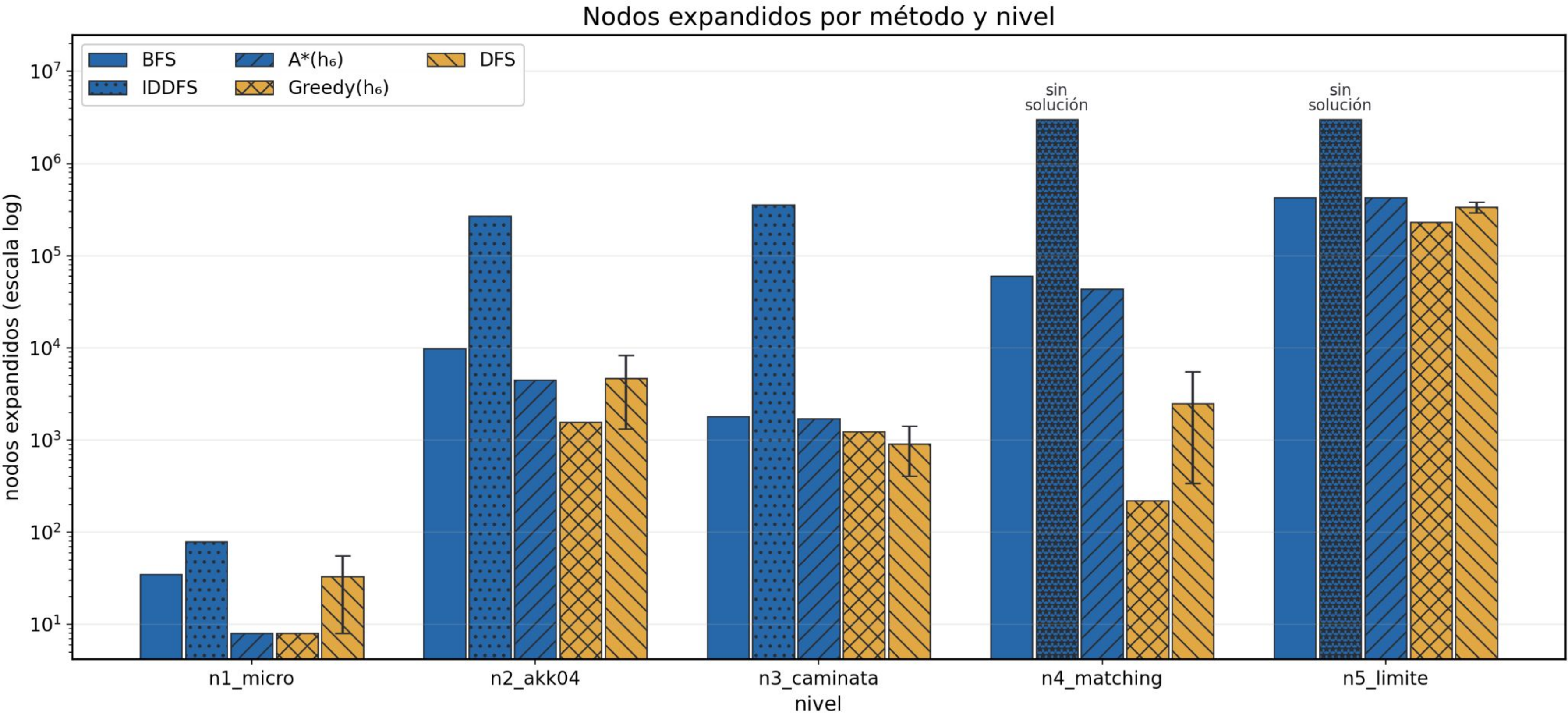


N4 · 70 mov



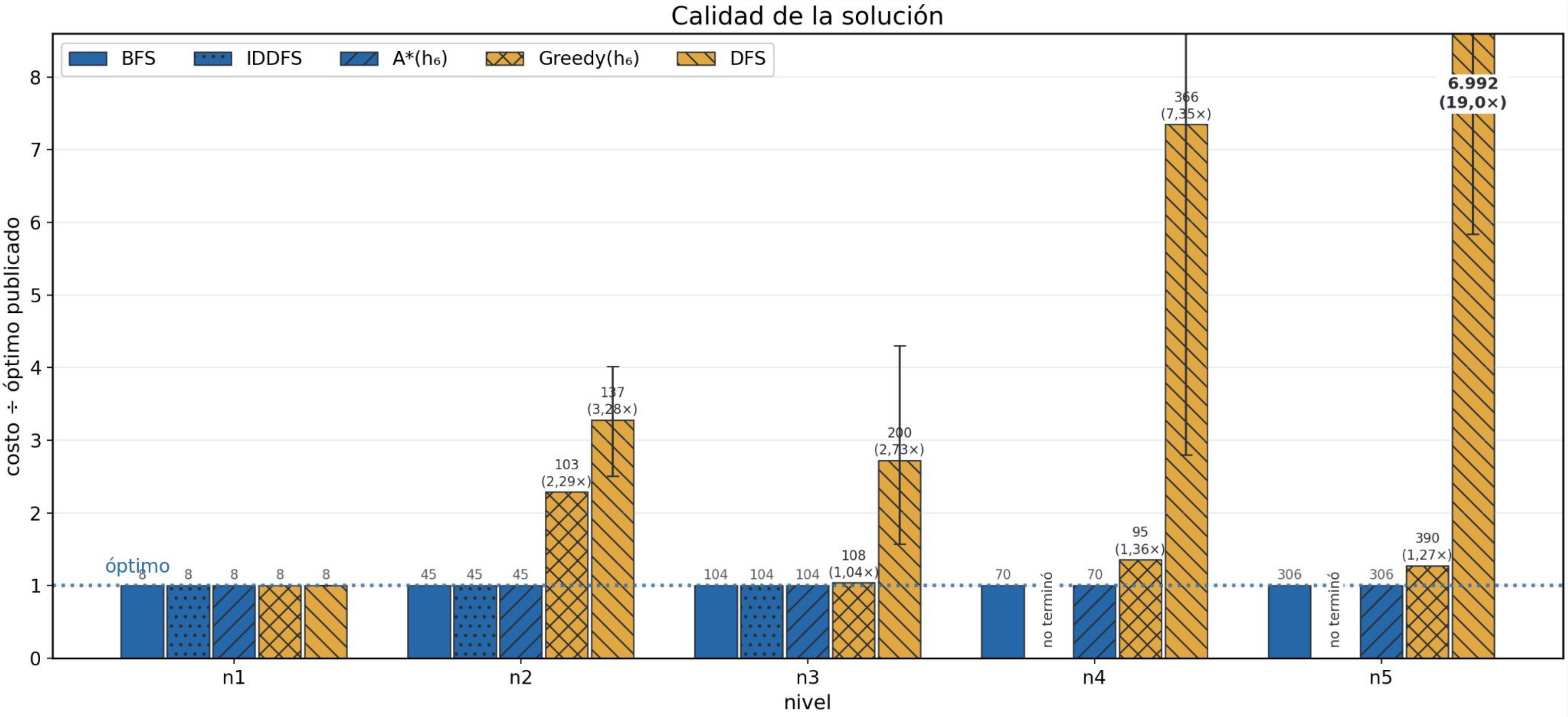
N5 · 306 mov

Nodos expandidos por método y nivel



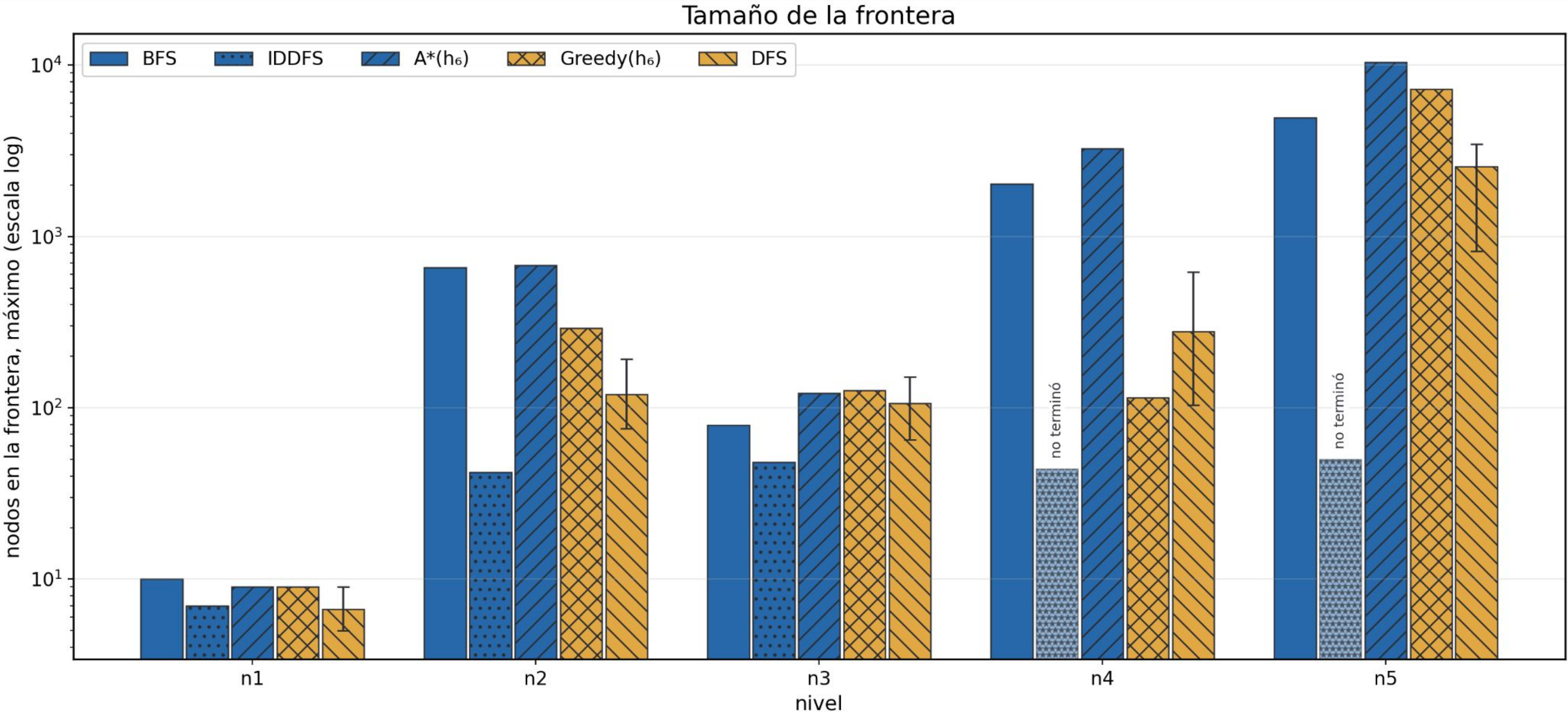
Escala logarítmica · azul devuelve el óptimo, arena no · bigotes en DFS: rango exacto de las 24 permutaciones

Calidad de la solución



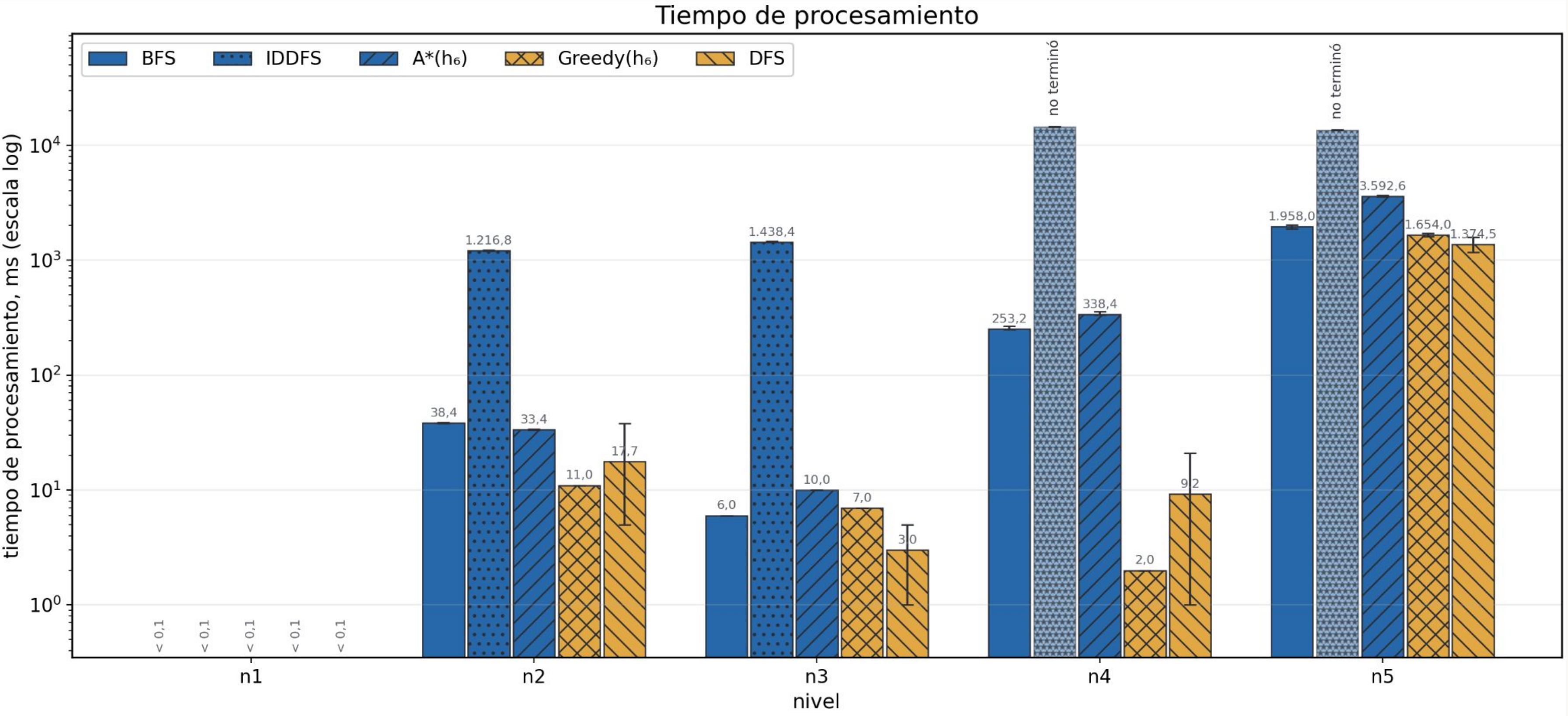
Costo ÷ óptimo publicado · BFS, IDDFS y A* siempre en 1 · Greedy entre 1,04x y 1,36x · DFS hasta 19x en N5

Tamaño de la frontera



Máximo de nodos en frontera, escala log · IDDFS gasta ~40 nodos donde BFS y A* llegan a 10⁴ en N5

Tiempo de procesamiento



Milisegundos, escala log · Greedy resuelve N4 en 2 ms y A* en 338 ms · IDDFS no termina en N4 ni N5

Todos los datos en un mismo lugar

nivel	método	resultado	costo (movimientos)	nodos expandidos	frontera máxima	memoria máxima	tiempo (ms)
n1_micro	BFS	ÉXITO	8	35	10	53	< 0,1
n1_micro	IDDFS	ÉXITO	8	79	7	30	< 0,1
n1_micro	A*(h ₆)	ÉXITO	8	8	9	26	< 0,1
n1_micro	Greedy(h ₆)	ÉXITO	8	8	9	26	< 0,1
n1_micro	DFS	ÉXITO	8	8-56	5-9	26-63	< 0,1
n2_akk04	BFS	ÉXITO	45	9.839	661	11.111	38,4
n2_akk04	IDDFS	ÉXITO	45	271.398	42	9.643	1.216,8
n2_akk04	A*(h ₆)	ÉXITO	45	4.460	677	5.793	33,4
n2_akk04	Greedy(h ₆)	ÉXITO	103	1.565	292	2.149	11,0
n2_akk04	DFS	ÉXITO	113-181	1.329-8.358	76-193	1.521-8.542	17,7
n3_caminata	BFS	ÉXITO	104	1.816	79	1.836	6,0
n3_caminata	IDDFS	ÉXITO	104	355.058	48	1.826	1.438,4
n3_caminata	A*(h ₆)	ÉXITO	104	1.702	122	1.778	10,0
n3_caminata	Greedy(h ₆)	ÉXITO	108	1.243	126	1.445	7,0
n3_caminata	DFS	ÉXITO	164-448	408-1.412	65-151	535-1.621	3,0
n4_matching	BFS	ÉXITO	70	60.410	2.028	62.306	253,2
n4_matching	IDDFS	FRACASO	cortado en 3.000.000	3.000.000	44	47.397	14.494,6
n4_matching	A*(h ₆)	ÉXITO	70	43.628	3.265	47.121	338,4
n4_matching	Greedy(h ₆)	ÉXITO	95	219	115	449	2,0
n4_matching	DFS	ÉXITO	196-1.158	339-5.533	104-622	539-6.762	9,2
n5_limite	BFS	ÉXITO	306	429.817	4.917	430.017	1.958,0
n5_limite	IDDFS	FRACASO	cortado en 3.000.000	3.000.000	50	46.949	13.513,0
n5_limite	A*(h ₆)	ÉXITO	306	426.808	10.434	427.601	3.592,6
n5_limite	Greedy(h ₆)	ÉXITO	390	231.220	7.246	245.450	1.654,0
n5_limite	DFS	ÉXITO	1.786-7.692	294.796-381.328	819-3.456	298.913-387.702	1.374,5

Conclusiones finales

¿Se pueden generalizar los usos de cada algoritmo?

A* con h_5	por defecto	El método óptimo escalable: tolera tableros complejos sin agotar recursos.
Greedy con h_5	tiempo real	Ahorra hasta un 99% del tiempo con soluciones aceptables: la opción cuando la velocidad prima sobre el camino perfecto.
BFS	línea de base	Útil como control en grafos unitarios chicos o medianos, donde evaluar una heurística cara cuesta más que explorar en anchura.
DFS	cumplir restricciones	Da una mayor utilidad cuando hay árboles sin ciclos donde todas las metas residen a profundidad fija y el camino no importa.
IDDFS	ramificación alta o RAM restringida	Es ineficiente en grafos densos debido a las re-expansiones masivas por caminos redundantes.

¡Gracias!